
Integração e Entrega Contínua – DSM – Professora Lucineide – 1ºSemestre/2026**Atividade Prática – Aula 5****Tema – Testes Automatizados com Jest no CI**

Entrega da Atividade

1. Exporte/salve o arquivo de acordo com o padrão: **IEC-Atividade-Aula05.pdf**
2. Enviar a Atividade em PDF no GitHub:
 1. Acesse seu repositório da disciplina: **FATEC-JCR-4DSM-IEC-2026-1-seunome**
 2. Se ainda não existir, você deverá criar a pasta da atividade: **P1-Conteudos/Atividades/Atividade-Aula05/README.md**
 3. Faça o **commit** do pdf da atividade feita na pasta.
 4. Em seguida copie o **link** do pdf.
 5. Acesse o **Kanban-IEC-2026-1** da disciplina que se encontra no repositório da professora.
 6. Clique no card: **IEC-Atividade-Aula05**.
 7. No comentário, cole o **link** do **pdf** da atividade que você fez.
 8. Volte ao seu repositório e acesse o seu **Kanban-IEC-2026-1**.
 9. Mova o card **IEC-Atividade-Aula05** para a coluna **Entregue**.
3. Ou, no caso de configurações feitas no repositório do GitHub, enviar o link.

Exercício 1 – Instalar o Jest**Explicação (porquê):**

Precisamos instalar o framework para rodar testes.

Passo a passo (como fazer):

No terminal, na raiz do projeto:

`npm install jest --save-dev`

Adicione no package.json:

```
"scripts": {  
  "test": "jest"  
}
```

-> Agora podemos rodar **npm test** para executar os testes.

Integração e Entrega Contínua – DSM – Professora Lucineide – 1ºSemestre/2026**Exercício 2 – Criar primeiro teste****Explicação (porquê):**

Vamos começar com um teste simples para entender como o Jest funciona.

Passo a passo (como fazer):

Crie a pasta tests.

Dentro dela, crie soma.test.js com:

```
function soma(a, b) {  
  return a + b;  
}
```

```
test("soma de 2 + 3 deve ser 5", () => {  
  expect(soma(2, 3)).toBe(5);  
});
```

Rode no terminal:

```
npm test
```

O Jest deve mostrar: **Test passed.**

Exercício 3 – Criar teste aplicado ao ABP**Explicação (porquê):**

Testar uma função realista ajuda a conectar teoria e prática.

Passo a passo (como fazer):

Crie tests/alerta.test.js com:

```
function classificarAlerta(nivel) {  
  if (nivel > 80) return "Crítico";  
  if (nivel > 50) return "Alto";  
  return "Moderado";  
}
```

```
test("alerta crítico se nível maior que 80", () => {  
  expect(classificarAlerta(90)).toBe("Crítico");  
});
```

```
test("alerta alto se nível entre 51 e 80", () => {
```

Integração e Entrega Contínua – DSM – Professora Lucineide – 1ºSemestre/2026

```
expect(classificarAlerta(70)).toBe("Alto");  
});
```

```
test("alerta moderado se nível até 50", () => {  
  expect(classificarAlerta(30)).toBe("Moderado");  
});
```

Rode **npm test**.

Os três testes devem passar.

Exercício 4 – Integrar Jest no GitHub Actions**Explicação (porquê):**

Agora o CI/CD garante que apenas código com testes aprovados seja aceito.

Passo a passo (como fazer):

1. Edite `.github/workflows/ci.yml`:

name: CI com ESLint e Testes

on:

push:

branches: [dev]

pull_request:

branches: [dev]

jobs:

lint-and-test:

runs-on: ubuntu-latest

steps:

- name: Checkout do repositório

uses: actions/checkout@v4

- name: Configurar Node.js

uses: actions/setup-node@v4

with:

Integração e Entrega Contínua – DSM – Professora Lucineide – 1ºSemestre/2026

node-version: '18'

- name: Instalar dependências

run: npm install

- name: Rodar ESLint

run: npx eslint .

- name: Rodar Jest

run: npm test

2. Faça commit.

3. Vá em **Actions** -> confirme que o pipeline agora roda **ESLint + Jest**.

Exercício 5 – Simular falha de teste**Explicação (porquê):**

Ver como o pipeline reage quando um teste falha é essencial.

Passo a passo (como fazer):

1. No arquivo alerta.test.js, altere o teste:

```
test("alerta crítico se nível maior que 80", () => {  
  expect(classificarAlerta(90)).toBe("Alto");  
});
```

2. Rode **npm test** -> o teste falhará.

3. Faça commit e abra PR.

4. O pipeline no GitHub Actions deve falhar.

5. Volte o teste ao correto e faça novo commit.